



PDF Download
3502718.3524776.pdf
10 February 2026
Total Citations: 4
Total Downloads: 135

Latest updates: <https://dl.acm.org/doi/10.1145/3502718.3524776>

RESEARCH-ARTICLE

POGIL-like Learning in Undergraduate Software Testing and DevOps - A Pilot Study

BHUVANESWARI GOPAL, University of Nebraska–Lincoln, Lincoln, NE, United States

STEVE COOPER, University of Nebraska–Lincoln, Lincoln, NE, United States

Open Access Support provided by:

University of Nebraska–Lincoln

Published: 07 July 2022

[Citation in BibTeX format](#)

ITiCSE 2022: Innovation and Technology
in Computer Science Education
July 8 - 13, 2022
Dublin, Ireland

Conference Sponsors:
SIGCSE

POGIL-like Learning in Undergraduate Software Testing and DevOps - A Pilot Study

Bhuvaneswari Gopal
bhuvana.gopal@unl.edu
University of Nebraska-Lincoln
Lincoln, Nebraska, USA

Stephen Cooper
stephen.cooper@unl.edu
University of Nebraska-Lincoln
Lincoln, Nebraska, USA

ABSTRACT

Process Oriented Guided Inquiry-based Learning (POGIL) is an active learning pedagogy that emphasizes process skills in addition to content, and has been shown to improve student learning outcomes in various STEM disciplines. We developed 4 POGIL-like activities for unit testing, integration testing and continuous integration, in software engineering. We delineate one of the activities in this paper, and explain how we implemented our POGIL-like pedagogy in the classroom. We measured student success through a cognitive pre- and post-course survey for those topics, and found significant increases in score correctness gains. We also document and hypothesize reasons for the cognitive gains from the POGIL-like pedagogy.

CCS CONCEPTS

• **Software and its engineering**; • **Social and professional topics** → **Computing education**;

KEYWORDS

software engineering education, software testing, DevOps, continuous integration, unit testing, integration testing, pedagogy, inquiry based learning, process oriented guided inquiry based learning

ACM Reference Format:

Bhuvaneswari Gopal and Stephen Cooper. 2022. POGIL-like Learning in Undergraduate Software Testing and DevOps - A Pilot Study. In *Proceedings of the 27th ACM Conference on Innovation and Technology in Computer Science Education Vol 1 (ITiCSE 2022)*, July 8–13, 2022, Dublin, Ireland. ACM, New York, NY, USA, 7 pages. <https://doi.org/10.1145/3502718.3524776>

1 INTRODUCTION

Software testing helps ensure software quality and reliability. Lemos et al. [37] investigated the impact of software testing knowledge on the production of reliable code. They conducted a controlled experiment involving 28 senior-level computer science students, results showed that there was on average, 20% more reliability in the code after students were taught how to test. Edwards [16] explains that many undergraduate programs lack real practical training on how to test their code and often have poor software testing skills

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

ITiCSE 2022, July 8–13, 2022, Dublin, Ireland

© 2022 Association for Computing Machinery.

ACM ISBN 978-1-4503-9201-3/22/07...\$15.00

<https://doi.org/10.1145/3502718.3524776>

[16], and this underscores the need to pay more attention to testing and to improve the effectiveness of education in the testing field [6]. One reason for this problem might be that teaching software testing has not kept pace with the rapidly changing field of software testing [49]. It is also difficult to teach software testing [8]. Students often find learning software testing challenging [19]. Identifying entry and exit points into test cases, understanding what to test and at what level of granularity, and tying test cases to requirements, were some of the important difficulties uncovered by Gopal and Cooper in their study on student difficulties in learning software testing and DevOps [19].

To help alleviate some of the issues mentioned above, active learning pedagogies have become popular in CS education in recent years. POGIL [34] is one such pedagogical approach involving teamwork and a well defined process. It has been shown to improve student learning in STEM disciplines as well as entry level CS courses [35]. In this paper, we present the results of employing a POGIL-like pedagogy in an undergraduate software engineering classroom in the topics of unit testing, integration testing and continuous integration. Due to trademark reasons that forbid the usage of the term POGIL unless officially approved by the POGIL Project [1], we utilize the term "POGIL-like" to denote our pedagogical approach throughout this paper.

1.1 Bird's eye view of the POGIL-like approach

POGIL utilizes two fundamental concepts: active learning, and collaborative learning. Traditional lecturing does not leverage the collective human characteristics of the classroom, which POGIL strives to utilize. POGIL-like, similar to POGIL [34], is a process heavy approach, with multiple roles, artifacts and processes governing its implementation.

In the POGIL-like pedagogy, students are organized into small teams. Students start each class session with little or no prior knowledge of the topic, so they can benefit from the co-construction of knowledge through POGIL-like activities without added misconceptions. The instructor serves as an active facilitator, walking around the classroom and helping students as needed during the session. Each team consists of 4-6 students, and each student has a specific role to play. There are 4 roles: Manager, Recorder, Presenter and Reflector. Students engage with "models" and "activities". Models are a compilation of content knowledge that the students need to master. Models typically contain figures, tables, equations, and code snippets in addition to plain text. Activities contain critical thinking questions and hands on exercises. These questions and exercises must be robust and well-aligned with specific objectives, both from a content perspective as well as a process perspective. These models and activities are designed to clearly define what the

student must learn, and provide them a pathway to construct the required knowledge based on active exploration and synthesis. This distinguishes POGIL (and hence, POGIL-like) from simply having students complete worksheets in small groups [43].

After working through the models and activities, the student designated as the Presenter provides a summary of what the team learned during the session, and the instructor discusses the findings with the whole class. The way that students build their knowledge in POGIL is by exploring the models to "invent" important concepts and eventually apply what they learned [22] through the co-operative, role-based interactions that they have with fellow students within their small groups. This knowledge exploring, concept inventing and application process can be utilized for both technical content and process-related content (e.g. problem solving, teamwork, and written/oral communication) [26].

The rest of the paper is organized as follows. We present prior work in software testing and in POGIL in Section 2. We present our research question in Section 3. Our research methods are explained in Section 4, which includes a detailed explanation of how we implemented the POGIL-like pedagogy in our classroom. Results are presented in Section 5, with a discussion of results in Section 6. We detail the threats to the validity of our study in Section 7, and conclude in Section 8.

2 PRIOR WORK

Several studies explore software testing education, specifically utilizing lecture-alternate teaching approaches. Coutinho et al. [11] studied online software testing education in undergraduate CS courses. They utilized the Problem-Based Learning (PBL) approach along with Just-in-time-teaching (JiTT), and results show that students positively perceived the inclusion of an active methodology in the learning process. Smith et al. 2012 [46] used peer review to teach software testing. The study evaluated students' performance through a survey summarizing student attitudes taken throughout the course. Fraser et al. [17], integrated a game called Code Defenders as a semester-long activity of an undergraduate and graduate course on software testing.

Machemer [39] documented the advantages of active and cooperative learning. Other active and collaborative learning techniques such as pair programming have been used in CS education for a few years [14, 41], with the "flipped classroom" or "inverted classroom" now gaining popularity [40]. The adoption of evidence-based instructional practices among undergraduate science, technology, engineering, and mathematics (STEM) faculty members has been a focus in current instructional reforms in STEM courses [10, 21].

2.1 POGIL: The origins

Over two decades, POGIL has been studied in various STEM disciplines, particularly in chemistry and biomechanics [5, 23, 43, 45, 47]. POGIL was originally implemented in college general chemistry courses, with promising improvements in student performance, in addition to reducing DWF rates [22, 43]. Various research questions that have been asked in the context of POGIL include facilitation of collaborative learning [23], academic performance and academic confidence [13], efficiency in large classrooms [5] and the role of

instructor intervention [12]. Pioneering work on POGIL in CS has been done by Kussmaul [34] and Hu [27].

2.2 POGIL in comparison with other student centered pedagogies

Social constructivism [7] is the underlying principle behind POGIL [35]. Compared to most other active learning strategies, POGIL implementation clearly relies on constructivist concepts, by using specific ways in which models and activities are designed and facilitated. POGIL uses an explore-invent-apply learning cycle [3] based on a Piagetian model of learning further expounded by Karplus [33].

POGIL exhibits common characteristics with other forms of inquiry-based learning [15]. POGIL is often compared with Problem-Based Learning (PBL) and Peer-Led Team Learning (PLTL). PBL organizes students in larger groups and assigns more complex and open-ended problems with less formal guidance than POGIL activities [20, 28]. PLTL differs significantly from POGIL in that a more experienced student (someone who has recently completed the course, perhaps similar to a teaching assistant), rather than the instructor, serves as the facilitator, leading students in activities and discussions to improve their understanding of the material [28, 31]. A comprehensive comparison on the similarities and differences among PBL, POGIL, and PLTL can be found in the study by [15].

2.3 POGIL in introductory CS courses

In CS, POGIL has been mainly studied in introductory CS classes [25, 27, 35, 36]. Computer science professors at Furman University used inquiry-based learning in five introductory CS classes, including CS1 and CS2. Their implementation differed from POGIL in that students had preparatory individual assignments. The instructors created inquiry-based assignments for CS2 [2]. Medley [42] compared inquiry based learning with pure lectures in two different sections of the same CS1 class. Their implementation differed from pure POGIL in that student groups were not mandatory. They found a higher pass rate for the inquiry-based section than the lecture section, but also higher withdrawal rates.

2.4 POGIL in Cybersecurity

The POGIL approach has been gaining popularity in cybersecurity education. Yang et al. [51] developed a POGIL activity for teaching access control. Alshaher et al. [4] developed POGIL activities for the topic of software defined network data plane flooding attacks. They implemented the activities in their advanced security for emerging networks course and reported positive student feedback. Yuan et al. [52] reported their experience of developing GICL (Guided Inquiry Collaborative Learning) activities for firewall and IPsec. They found that the GICL activity increased student grades by 20.2%.

2.5 POGIL in SE

In addition to the introductory CS0 and CS1 courses, Kussmaul has introduced POGIL to several introductory and mid level CS classes, including CS2 and software engineering [34, 35]. There have also been "structured like POGIL" activities in other introductory CS classes [9]. Kussmaul [35] emphasizes that POGIL is especially useful for teaching important skills such as problem-solving and

teamwork. Many studies of POGIL in CS are focused on CS1 [28, 29, 32], along with faculty motivation and challenges [50]. Some POGIL activities have been developed in process related aspects of software engineering [36]. More recently, POGIL has been demonstrated to help teach design patterns effectively [38].

Using POGIL in CS poses a unique set of challenges [34]. Faculty need to invest significant time and effort developing them, since POGIL is so process-heavy. CS courses and curricula have a wide variety of rapidly changing content. This makes it more difficult to share and utilize materials from other institutions. Currently, the POGIL activity bank for CS, particularly SE, is sparse. A comprehensive set of POGIL activities have been developed and published for introductory CS classes as well as some process-focused topics in SE [24, 26, 28, 29, 34, 35]. As we can see, the current activities in SE are primarily process based- such as project scheduling, software development life cycles, software release life cycles, version control, and story point estimation. There is a dearth of literature on how POGIL can be utilized in a software engineering classroom for technical content, in software quality topics such as unit testing, integration testing or continuous integration. Our focus in this pilot study was to examine how a POGIL-like pedagogy would potentially influence student learning outcomes for technical content instruction in the topics of software testing and DevOps.

3 RESEARCH QUESTION

In our study we aimed to address the following research question. **RQ:** Does incorporation of the POGIL-like pedagogy help students in an undergraduate SE class learn the concepts of unit testing, integration testing and continuous integration better than students learning in a purely lecture format class?

4 METHODS

4.1 Study Context

Our study was reviewed and determined to be exempt by our university's Institutional Review Board. The instructor of the course had attended multiple POGIL workshops conducted through the POGIL project [1], including an activity writing workshop. We created 4 POGIL-like activities for this study, 2 in unit testing, and 1 each in integration testing and continuous integration.

Student groups: In this study, we targeted 60 students enrolled across two sections of a semester long sophomore level software engineering (SE) course at a large R1 university. One section was taught using lectures, and we called this section the control group (CG). The other section was taught using the POGIL-like pedagogy, and we called this section treatment group (TG). Students did not opt into a POGIL-like/lecture version. Both sections of the class were taught by the same instructor with the same content in the same order, in classes one hour apart. We employed a pre- post-test quasi experimental study design, and analyzed the variance in our data between and within both groups. This design is to limit and avoid a potential selection bias in our sample. Our student population primarily majored in computer science and computer Engineering.

Class sessions: Each section of the course consisted of three 50-minute class sessions per week over 16 weeks. We implemented the POGIL-like pedagogy using a POGIL learning cycle [34], in 4

class sessions as follows: 2 sessions in unit testing, 1 for integration testing, and 1 for continuous integration.

Before each class session: POGIL-like is a constructivist pedagogy, and we wanted students to come into class ready to explore and learn. Therefore, we ensured that no readings or prior preparation is given to them on the topic.

During class: POGIL-like activities in small groups (4-6 students) (30 minutes): Students were assigned to small groups, and participated in activities crafted to guide them through the exploration of challenging concepts [35]. Students participated in a POGIL-like cycle for each topic, shared their understanding and reasoning with each other, and learn the topic, while recording salient concepts that they learned.

At the end of each class: Report- out with instructor led discussion (5-10 minutes): The instructor lead a class-wide discussion. Students from all teams shared their thought process and POGIL-like group discussions. The instructor provided clarifications for lingering misconceptions and questions, and clearly indicated the correct answers for each question in the activities.

At the beginning of the semester, we surveyed the students in both groups using a cognitive questionnaire that was published by Gopal and Cooper [18], that included 16 questions - 6 in unit testing, 3 in integration testing and 7 in continuous integration. All questions were multiple choice with one single correct answer. We surveyed the students again at the end of the semester, after they had undergone instruction using lectures or the POGIL-like pedagogy, using the same cognitive questionnaire. Students were not compensated for their participation in the surveys.

4.2 Student and instructor roles

Student roles: Our student teams consisted of 4 distinct roles [29]: Manager, Recorder, Presenter and Reflector. The Manager kept the team focused on the task, managing time throughout the POGIL-like class session. The Recorder recorded answers for the team. The Presenter presented answers (or questions) to the class and the instructor at the end of the session. The Reflector considered how the team was working and how it could improve in the future. For each POGIL-like session, we gave each team member a role card that indicates their role and its responsibilities. We randomly assigned a new team for each of the 4 sessions of our study, with care being taken to rotate roles for each session. For example, if a fictional student named Mary was a recorder on team 1 for POGIL session 1, she might be a Reflector on team 5 for session 2. This helped us to facilitate students gaining experience with all roles, and with different teams. These roles were assigned to each student to foster interdependence as well as individual responsibility and accountability to the success of the team [35].

Role of the instructor: In our classroom (30 students per classroom) the instructor was able to specifically assign groups and observe closely how students interacted with each other using the prescribed roles [28]. Simultaneously, the instructor focused on helping students develop process skills, specifically, problem solving, teamwork and critical thinking [28]. The instructor was quick to offer timely, additional guidance as students worked in teams [22, 34].

4.3 Activity development: Models, E-I-A cycles and D/C/V questions

A model in a POGIL-like activity denotes the content that we would provide for students to know, during lectures or required readings [40], but presented with action verbs, figures, pictures and tables as needed, instead of copious quantities of plain flowing text. We used a combination of three types of questions -Directed (D), Convergent (C) and Divergent (V) questions. Directed questions are based on prior knowledge or provided information and are rarely difficult to answer. Convergent questions require more effort, but most students or teams will reach similar answers. Divergent questions also require more effort, but students and teams will often reach very different answers and explore broader issues [1]. Each of these D/C/V questions were intended to encourage the students towards either Exploration (E), Concept Invention (I) or Application (A) [30].

In each of our POGIL-like activities, we included one or more models. We followed each model with an activity. Each activity contained some combination of E-I or E-A or I-A questions, which also correlated to the D/C/V classification. For example, an I/C question would indicate that it was a concept invention question that was convergent, an E/D question be an exploration question that was directed, and an A/V question would be an application question that was divergent. In addition, we included explicit Content Learning Objectives (CLO) and Process Learning Objectives (PLO) in each activity. An example POGIL-like activity is shown below. We designed each activity to focus on 3-4 technical concepts and 1-2 process skills, to be completed within 40 minutes.

4.4 POGIL-like in our classroom: an example activity

Here is an example excerpt from an E-I-A cycle in our POGIL-like classroom, utilizing D/C/V questions:

UNIT TESTING POGIL SESSION 1

CLO: Describe positive, negative and exception flows for unit testing, differentiate between unit tests and other tests, identify testable code, identify test conditions. **PLO:** information processing, problem solving, teamwork, critical thinking

start time:

Prerequisites: Proficiency in writing classes with multiple methods in C#.NET or any other Object oriented language framework; familiarity with database concepts.

MODEL 1: Unit tests isolate and exercise specific parts, also known as units of your code. A unit is typically a method in your code.

Activity: In the screenshot above, what is the unit of work that we would write unit tests for?

Answer: This is an E/D question, the answer is the divide(0) method.

MODEL 2:

ACTIVITY:

1) What is the test browser for both actions 1 and 2?

Answer: This is an E/D question, and the answer is IE-11.

2) What is the action in the second row?

Answer: This is an E/D question, and the answer is Enter correct email and password and hit login button

Figure 1: Model 1 - What is the unit of work here?

```
namespace Calculations
{
    public class Division
    {
        public static int DivideThis ( int numerator, int denominator)
        {
            int resultValue = numerator / denominator;
            return resultValue;
        }
    }
}
```

Figure 2: Model 2 - Test Cases

Test Scenario #	Logging in		Test Case #	Login1		
Description	Login-Success		Priority	High		
Pre-conditions	Valid user credentials		Post-conditions	None		
Serial	Action	Inputs	Expected output	Actual output	Browser	Result
#						Comments
001	Run and launch app	https://www.facebook.com	Facebook home	Facebook home	IE-10	Pass
002	Enter valid credentials and click on Login button	Email: testing@acb.com ; Password: *****	Login successful	Login successful	IE-10	Pass
						ABC, 12:30 pm on 12/22/21
						PQR, 12:45 pm on 12/22/21

3) What is the Actual output in row 1? Does it match the expected output in row 1?

Answer: This is an E/D question, and the answer is Facebook home. Yes it matches the expected output.

4) Based on the model above, consider the following statements, and describe whether each of these is a unit test or not. Give the rationale behind your choice.

A. Code that writes records to the database.

B. Test that creates thousands of requests for a service you've written.

C. A test that can fail when run on a machine without the "proper setup".

Answer:

This is a I/C question. All 3 of them are not unit tests. Rationale: A. Unit tests should not depend on external databases. B. Unit tests are not stress tests. C. Unit tests should not require "proper setup" and should simply test the method in isolation.

MODEL 3: The syntax to write a simple unit test can be found here (we provided the link to the C#.NET API here). Utilize it and please write a simple unit test for the code in Model 1.

(This is an A/C question). Students would write a unit test here.

As we can see above, we used a combination of D/C/V questions, building up from simple directed knowledge based questions, to questions that need progressively higher levels of deeper thinking and analysis, using convergent, and divergent questions. We moved through knowledge exploration (that an individual method is a unit of work), through concept invention (that unit tests are written a certain way, with the right conditions, and without external dependencies), and we ended with an application question, thus covering the E-I-A cycle within the POGIL-like session.

5 RESULTS

5.1 Data Analysis Techniques

We analyzed data from the pre- and post- surveys to see if there was a statistically significant difference in scores between groups (control group vs treatment group) as well as within groups (within control group pre and post, within treatment group pre and post). We used the Shapiro-Wilk test to see if the data was normally distributed. The test combines skew and kurtosis to produce an omnibus test of normality. We checked for homoscedasticity using Levene's test for homogeneity of variance across all values of the independent variables. Based on the results of these tests, we chose the appropriate ANOVA test (regular or Welch's) with associated p-value to determine the statistical significance of the results [48]. Throughout this paper, we use a p-value of less than 0.05 to indicate that the results found are significant. There were 30 students in the treatment group and 30 students in the control group. Our overall data were not normally distributed (Shapiro-Wilk normalcy test $p < 0.001$).

5.2 Starting points: Pre-control vs pre-treatment

In comparing learning outcomes on the basis of the scores on the pre survey for the control group versus the treatment group in each individual topic, we found that the two groups had similar starting points in the topic of unit testing (pre-test: $\chi^2 = 0.45$, $p = 0.22$), integration testing ($\chi^2 = 0.43$, $p = 0.31$) and continuous integration ($\chi^2 = 0.11$, $p = 0.65$). Students in both groups has starting points that were not statistically different. Analyzing the pre-survey scores for both groups for all 3 topics combined, yielded a similar result ($\chi^2 = 0.18$, $p = 0.32$).

5.3 Post control vs post treatment

We compared learning outcomes for the control group versus the treatment group on the basis of the post survey scores. We found that the two groups differed significantly on how much they knew, as evidenced by their overall post survey scores: ($\chi^2 = 44.45$, $p < 0.0001$). Our results of the Kruskal-Wallis tests for each category, at an α level of 0.05 are: unit testing ($\chi^2 = 42.24$, $p < 0.001$); integration testing ($\chi^2 = 44.38$, $p < 0.001$); and continuous integration ($\chi^2 = 44.13$, $p < 0.001$). Students in the POGIL-like class scored significantly higher than students in the purely lecture-based class. Figure 3 shows the distribution of scores for all 3 topics combined (overall scores) in the post-treatment group and post- control group.

5.4 Pre vs post treatment group

Based on our results, we found that students in the POGIL-like class (treatment group) scored significantly higher on their post survey than in the pre survey, as evidenced by their scores ($\chi^2 = 39.98$, $p < 0.0001$). Our results of the Kruskal-Wallis tests for each category, at an α level of 0.05 are: unit testing ($\chi^2 = 41.39$, $p < 0.001$); integration testing ($\chi^2 = 38.19$, $p < 0.001$); and continuous integration ($\chi^2 = 39.11$, $p < 0.001$). Figure 4 shows the distribution of scores in the pre and post-test for the treatment group.

Figure 3: Lecture vs POGIL-like, post survey

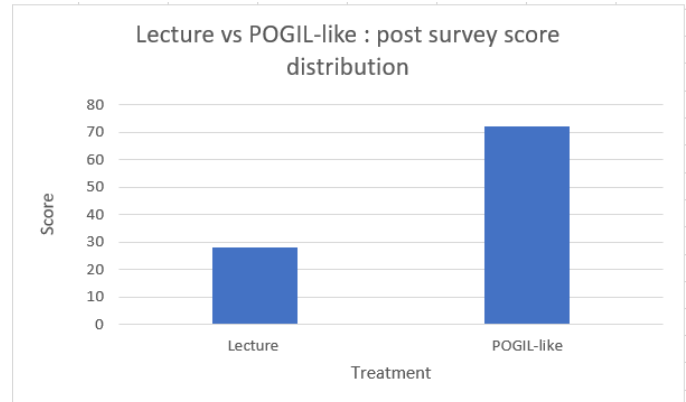
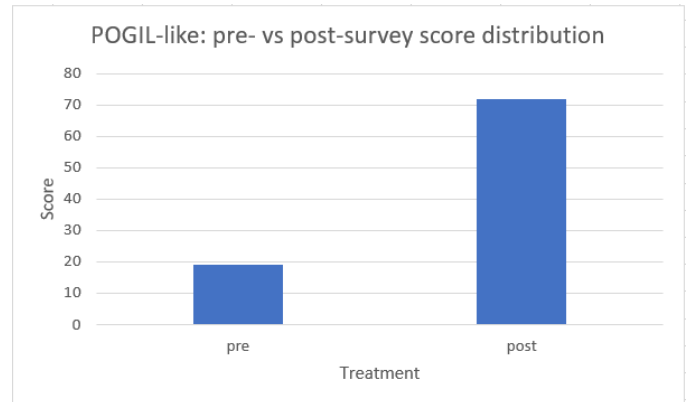


Figure 4: POGIL-like: pre vs post survey



6 DISCUSSION

We found that while both groups exhibited gains in all 3 categories as seen in Table 1. We define gain as follows:

Gain = (% of students who answered correctly in the post-test) - (% of students who answered correctly in the pre-test). We found that overall average scores increased from 19 to 65 for the treatment group. The control group had a noticeably smaller increase, from 18 to 37. Gains by the treatment group were also significantly larger than those of the control group in each of the 3 topics individually. Gains in unit testing were 37% (treatment) vs 17% (control); 50% (treatment) vs 17% (control) for integration testing; and 51% (treatment) vs 23% (control) for continuous integration, with an overall average correctness increase of 46%.

6.1 Breaking down the results

Both groups had been exposed to the unit testing topic in an earlier academic course. This could possibly explain the relatively high pre-test scores on Q1 and Q5. This prior content knowledge could have impacted their POGIL-like discussions during class, as well as their post test scores. The overall gains in unit testing (37) were lower for the treatment group compared to integration testing (50) and continuous integration (51).

Table 1: Percentage of students who answered each question correctly

Scores	Treatment		Control	
	Pre	Post	Pre	Post
unit testing				
Q1	27	60	30	50
Q2	20	57	23	43
Q3	20	63	17	40
Q4	27	67	23	37
Q5	33	67	30	43
Q6	17	50	20	33
Q1-Q6 Average	24	60.67	23.83	41
integration testing				
Q7	13	63	17	33
Q8	20	70	13	30
Q9	17	67	23	40
Q7-Q9 Average	16.67	66.67	17.67	34.33
continuous integration				
Q10	10	77	17	50
Q11	17	70	13	40
Q12	20	83	10	43
Q13	10	77	10	47
Q14	20	80	23	37
Q15	20	87	10	43
Q16	17	83	13	40
Q10-Q16 Average	16	68	14	37
Overall Average	19	65	18	37

Not surprisingly, we found the largest gains in integration testing and continuous integration. It is possible that in these topics, students had very little prior knowledge, since the courses they had undergone in the academic preparation leading up to this SE course did not include these topics.

Similar to the study with Peer Instruction by Gopal and Cooper [18], our survey contained questions examining how coupling and testability are related (Q5), and the link between software design (Q6), and software architecture (Q7, Q8) with unit and integration testing. Questions about test doubles (Q7, Q8) required an ability to synthesize implementation variations of testing doubles with concepts from database modeling [18]. We chose questions that had enough open-endedness to facilitate lively discussions during POGIL-like sessions. We believe that the benefits of these small group discussions, weaved together using a carefully crafted process that involves distinct roles and learning cycles, reflected in these score correctness percentage gains.

7 THREATS TO VALIDITY

Several uncontrollable and unknown factors that play into student performance in a class. Students are generally accustomed to

standard instruction models [44]. In the context of this study, we utilized a heavily process oriented POGIL-like pedagogy, which students had not encountered before this pilot study. We do not know how much of an impact their unfamiliarity with the pedagogy potentially influenced their learning outcomes. In spite of the best facilitation efforts by the instructor (who was trained in POGIL facilitation), it is hard to determine if students followed the process with full fidelity, and whether the pedagogy improved their affect in relation to software testing. We intend to understand how students connected with the pedagogy in future iterations of the course.

Another threat to the validity of this study is the fact that we collected data from two sections of a single course, with a small number of students in each iteration (treatment $n=30$, control $n=30$). We intend to repeat this pilot study with other SE courses and with larger numbers of students in the future to better establish the repeatability of our findings.

8 CONCLUSION

In this paper, we have presented findings from a quasi-experimental pilot study in SE, with a specific focus on the application of a POGIL-like pedagogy in teaching technical content for unit testing, integration testing and continuous integration. We saw significant score gains in the POGIL-like class in all three areas, and maximum gains in integration testing and continuous integration. The lecture class did not exhibit learning gains that were statistically significant. In this context, we believe that POGIL-like has a positive role to play in impacting student learning in these topics.

In response to the RQ: We found that our POGIL-like pedagogy does help undergraduate students in an SE class learn the concepts of unit testing, integration testing and continuous integration better than their counterparts in a purely lecture format class. This being a pilot study, we wish to underscore the need for replication to verify our findings from this study.

We have identified several areas for future work. How do students feel about utilizing POGIL-like to learn technical content? How do the process learning objectives tie in with content learning objectives in software testing as a whole? What was the specific role of POGIL-like discussions in attaining the cognitive correctness gains that we obtained in this study? We intend to address some of these questions in our future work.

REFERENCES

- [1] [n. d.]. CS-POGIL | DCV (Directed, Convergent, Divergent) Questions. [https://csPOGIL.org/DCV+\(Directed,+Convergent,+Divergent\)+Questions](https://csPOGIL.org/DCV+(Directed,+Convergent,+Divergent)+Questions)
- [2] K. Abernethy, P. Gabbert, and K. Treu. 1998. Inquiry-based computer science instruction: Some initial experiences. *ACM SIGCSE Bulletin* 30, 3, 14–17.
- [3] M. R. Abraham. 2005. Inquiry and the learning cycle approach. *Chemists' guide to effective teaching* 1, 41–52.
- [4] H. Alshaher, X. Yuan, and S. Khorsandroo. 2020. Teaching flooding attack to the SDN data plane with POGIL. In *Proceedings of the 21st Annual Conference on Information Technology Education*. 194–199.
- [5] C. P. Bailey, V. Minderhout, and J. Loertscher. 2012. Learning transferable skills in large lecture halls: Implementing a POGIL approach in biochemistry. *Biochemistry and Molecular Biology Education* 40, 1, 1–7.
- [6] L. Bijlsma, H. Passier, H. Pootjes, S. Stuurman, and N. Doorn. 2020. *How do students test software units? Part one: Their natural attitude diagnosed*. Technical Report. Open Universiteit, Faculty of Science, Department of Computer Science.
- [7] George M Bodner. 1986. Constructivism: A theory of knowledge. *Journal of Chemical Education* 63, 10, 873.
- [8] P.J. Clarke, J. Pava, D. Davis, F. Hernandez, and T.M. King. 2012. Using WRESTT in SE courses: An empirical study. In *Proceedings of the 43rd ACM Technical Symposium on Computer Science Education (SIGCSE '12)*, ACM, New York, NY, USA, 307–312.
- [9] B. Coleman and M. Lang. 2012. Collaboration across the curriculum: a disciplined approach to developing team skills. In *Proceedings of the 43rd ACM Technical Symposium on Computer Science Education*. 277–282.
- [10] National Research Council et al. 2012. A framework for K-12 Science Education: Practices, crosscutting concepts, and core ideas. National Academies Press.
- [11] J. Coutinho, W. Andrade, and P. Machado. 2021. Teaching Exploratory Tests through PBL and JiTT: An experience report in a context of distributed teams. In *Brazilian Symposium on Software Engineering*. 205–214.
- [12] P.L. Daubenmire, D.M. Bunce, C. Draus, M. Frazier, A. Gessell, and M.T. van Opstal. 2015. During POGIL implementation the professor still makes a difference. *Journal of College Science Teaching* 44, 5, 72–81.
- [13] L.N. de Gale, S. and Boisselle. 2015. The Effect of POGIL on Academic Performance and Academic Confidence. *International Council of Associations for Science Education* 26, 56–61.
- [14] C.A. de Lima Salge and N. Berente. 2016. Pair Programming vs. Solo Programming: What Do We Know After 15 Years of Research?. In *49th Hawaii International Conference on System Sciences (HICSS)*. IEEE, 5398–5406.
- [15] T. Eberlein, J. Kampmeier, V. Minderhout, R.S. Moog, T. Platt, P. Varma-Nelson, and H.B. White. 2008. Pedagogies of engagement: a comparison of PBL, POGIL, and PLTL. *Biochem. Mol. Biol. Educ* 36, 262–273.
- [16] S.H. Edwards. 2004. Using software testing to move students from trial-and-error to reflection-in-action. In *Proceedings of the 35th SIGCSE Technical Symposium on Computer Science Education*. 26–30.
- [17] G. Fraser, A. Gambi, M. Kreis, and J. Rojas. 2019. Gamifying a software testing course with code defenders. In *Proceedings of the 50th ACM Technical Symposium on Computer Science Education*. 571–577.
- [18] B. Gopal and S. Cooper. 2021. Peer Instruction in software testing and continuous integration. In *Proceedings of the 52nd SIGCSE Technical Symposium on Computer Science Education*. 255–260.
- [19] B. Gopal, S. Cooper, J. Olmanson, and R. Bockmon. 2021. Student difficulties in unit testing, integration testing and continuous integration: An exploratory pilot qualitative study. *Proceedings of the 32nd Annual Psychology of Programming Interest Group Workshop (PPiG)*.
- [20] T. Greening, J. Kay, J.H. Kingston, and K. Crawford. 1997. Results of a PBL trial in first-year Computer Science. In *Proceedings of the 2nd Australasian Conference on Computer Science Education*. 201–206.
- [21] D. Handelsman, J. Ebert-May, R. Beichner, P. Bruns, A. Chang, R. DeHaan, J. Gentile, S. Lauffer, J. Stewart, S.M. Tilghman, et al. 2004. *Scientific teaching*. Vol. 304. American Association for the Advancement of Science. 521–522 pages.
- [22] D.M. Hanson. 2005. Designing process-oriented guided-inquiry activities. *Pacific Crest*, 1–6.
- [23] S.M. Hein. 2012. Positive impacts using POGIL in organic chemistry. *Journal of Chemical Education* 89, 7, 860–864.
- [24] H.H. Hu and B. Avery. 2015. CS principles with POGIL activities as a learning community. *Journal of Computing Sciences in Colleges* 31, 2, 79–86.
- [25] H.H. Hu and C. Kussmaul. 2012. Promoting student-centered learning with POGIL. In *Proceedings of the 43rd ACM Technical Symposium on Computer Science Education*. 579–580.
- [26] H.H. Hu, C. Kussmaul, B. Knaeble, C. Mayfield, and A. Yadav. 2016. Results from a survey of faculty adoption of process oriented guided inquiry learning (POGIL) in Computer Science. In *Proceedings of the 2016 ACM Conference on Innovation and Technology in Computer Science Education*. 186–191.
- [27] H.H. Hu, M. Lang, C. Kussmaul, C. Mayfield, and T. Pirmann. 2014. Guided inquiry learning in context: perspectives on POGIL in CS. In *Proceedings of the 45th ACM Technical Symposium on Computer Science Education*. 467–468.
- [28] H.H. Hu and T.D. Shepherd. 2013. Using POGIL to help students learn to program. *ACM Transactions on Computing Education (TOCE)* 13, 3, 1–23.
- [29] H.H. Hu and T.D. Shepherd. 2014. Teaching CS 1 with POGIL activities and roles. In *Proceedings of the 45th ACM Technical Symposium on Computer Science Education*. 127–132.
- [30] Minjie Hu, Michael Winikoff, and Stephen Cranefield. 2012. Teaching novice programming using goals and plans in a visual notation. In *Proceedings of the Fourteenth Australasian Computing Education Conference-Volume 123*. 43–52.
- [31] S. Huss-Lederman, D. Chinn, and J. Skrentny. 2008. Serious fun: Peer-led team learning in CS. In *Proceedings of the 39th SIGCSE Technical Symposium on Computer Science Education*. 330–331.
- [32] M. Jonas. 2019. Lessons learned from integrating POGIL into a CS1 course. *Journal of Computing Sciences in Colleges* 34, 6, 139–140.
- [33] R. Karplus and H.D. Thier. 1967. *A new look at elementary school science: Science curriculum improvement study*. Rand McNally.
- [34] C. Kussmaul. 2011. Process oriented guided inquiry learning for soft computing. In *International Conference on Advances in Computing and Communications*. Springer, 533–542.
- [35] C. Kussmaul. 2012. Process oriented guided inquiry learning (POGIL) for Computer Science. In *Proceedings of the 43rd ACM Technical Symposium on Computer Science Education*. 373–378.
- [36] C. Kussmaul, C. Mayfield, and H.H. Hu. 2017. Process oriented guided inquiry learning in Computer Science: The CS-POGIL & IntroCS-POGIL projects. In *ASEE Annual Conference and Exposition, Conference Proceedings*. 1–7.
- [37] O. Lemos, F. Ferrari, F. Silveira, and A. Garcia. 2015. Experience report: Can software testing education lead to more reliable code? In *2015 IEEE 26th International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, 359–369.
- [38] P.C. Lotlikar and R. Wagh. 2016. Using POGIL to teach and learn design patterns—a constructionist based incremental, collaborative approach. In *2016 IEEE Eighth International Conference on Technology for Education (T4E)*. IEEE, 46–49.
- [39] P.L. Machemer and P. Crawford. 2007. Student perceptions of active learning in a large cross-disciplinary classroom. *Active learning in higher Education* 8, 1, 9–30.
- [40] M. L. Maher, C. Latulipe, H. Lipford, and A. Rorrer. 2015. Flipped classroom strategies for CS Education. In *Proceedings of the 46th ACM Technical Symposium on Computer Science Education*. 218–223.
- [41] C. McDowell, L. Werner, H.E. Bullock, and J. Fernald. 2006. Pair programming improves student retention, confidence, and program quality. *Commun. ACM* 49, 8, 90–95.
- [42] M. D. Medley. 2007. Inquiry-based learning in CS1. *Journal of Computing Sciences in Colleges* 23, 2, 209–215.
- [43] R.S. Moog and J.N. Spencer. 2008. *Process oriented guided inquiry learning*. Vol. 994. American Chemical Society Washington, DC.
- [44] B. Simon, Päivi Kinnunen, L. Porter, and Dov Zazkis. 2010. Experience report: CS1 for majors with media computation. In *Proceedings of the 15th Annual Conference on Innovation and Technology in Computer Science Education*. 214–218.
- [45] S.R. Simonson and S. Shadle. 2013. Implementing process oriented guided inquiry learning (POGIL) in undergraduate biomechanics: Lessons learned by a novice. *Journal of STEM Education: Innovations and Research* 14, 1.
- [46] J. Smith, J. Tessler, E. Kramer, and C. Lin. 2012. Using peer review to teach software testing. In *Proceedings of the 9th Annual International Conference on International Computing Education Research*. 93–98.
- [47] A. Straumanis and E.A. Simons. 2008. A multi-institutional assessment of the use of POGIL in Organic Chemistry. ACS Publications.
- [48] R.L. Wasserstein and N.A. Lazar. 2016. The ASA statement on p-values: Context, process, and purpose. *Am. Stat* 70, 2, 129–133.
- [49] W. E. Wong, A. Bertolino, V. Debroy, A. Mathur, J. Offutt, and M. Vouk. 2011. Teaching software testing: Experiences, lessons learned and the path forward. In *2011 24th IEEE-CS Conference on Software Engineering Education and Training (CSEE&T)*. IEEE, 530–534.
- [50] A. Yadav, C. Kussmaul, C. Mayfield, and H.H. Hu. 2019. POGIL in computer science: Faculty motivation and challenges. In *Proceedings of the 50th ACM Technical Symposium on Computer Science Education*. 280–285.
- [51] L. Yang, X. Yuan, W. He, J. Ellis, and J. Land. 2019. Cybersecurity education with POGIL: Experiences with access control instruction. In *Journal of The Colloquium for Information Systems Security Education*, Vol. 6. 14–14.
- [52] X. Yuan, L. Yang, W. He, J.T. Ellis, J. Xu, and C.K. Waters. 2017. Enhancing cybersecurity education using POGIL. In *Proceedings of the 2017 ACM SIGCSE Technical Symposium on Computer Science Education*. 719–719.